

Insight Ladder — Day 1

How far does disorder spread?

Priyanka | September 21, 2026 | Easy relative to this week | 120 minutes

Main problem

Return the shortest inclusive interval $[l, r]$ whose sorting makes the entire integer array nondecreasing. Return $[-1, -1]$ if it is already sorted (including empty or singleton input). Do not modify the input. Arrays may contain duplicates; $n \leq 200,000$ and values lie between $-1,000,000,000$ and $1,000,000,000$. Require $O(n)$ time and $O(1)$ auxiliary space. First develop a slower baseline for small inputs.

Examples and checks: $[1, 3, 2, 2, 4]$ gives $[1, 3]$. $[2, 1]$ gives $[0, 1]$. $[2, 2, 2]$ gives $[-1, -1]$. Test a long ordered prefix and a small value near the end.

Optional variation

Instead of sorting, you may reverse one contiguous interval. Decide whether that can make the array nondecreasing, and return any valid interval, or IMPOSSIBLE. Already sorted arrays return $[-1, -1]$. An $O(n \log n)$ solution with $O(n)$ storage is acceptable for this optional task. Does the shortest sorting interval always work when reversed? Give a counterexample.

Before reading hints: Why might the first and last adjacent descents fail to identify the full interval? Explain why your final endpoints are necessary as well as sufficient.

Your 120 minutes

- 0–10: closed-notes recall of one invariant or failed assumption from the previous session.
- 10–100: solve the main problem. Write a direct solution, identify repeated work, state a claim, try to break it, then implement. Attempt the optional variation only when the main solution is secure; it stays inside these 90 minutes.
- 100–120: stop coding, explain correctness and cost, review evidence, and log partial work. A full solution read during review does not turn an unfinished timed attempt into a solve.
- Use C11 OR Python 3 consistently. After 15 minutes without a useful new step, record the blocker and request one hint or continue. Record the time and level of every hint or appendix consultation.

Submit your evidence

Submit code, tests, the key claim and its justification, runtime/space, minutes spent, and hints used. Main status: independent solve / assisted solve / partial / unattempted. Insight: independent / after small hint / after method supplied. Track variation coding or reasoning separately; a counterexample is not another code solve. Count a main solve only when working code meets the contract and you can justify it. There is no completion quota.

Solution appendix

Open only after an attempt; record what you read and when. Read one hint at a time. The explanation below supplies the method and cannot count as independent discovery.

Progressive hints

1. Compare the array with a sorted copy in your baseline. What do the first and last mismatching positions tell you?
2. When scanning left to right, what earlier value certifies that this position cannot stay outside the interval?
3. Maintain the maximum seen so far to find the last out-of-order position. Scan from the right with a running minimum to find the first.

Main solution reasoning

A sorted-copy baseline finds the first and last mismatches in $O(n \log n)$ time and $O(n)$ space. For the required method, scan left to right maintaining the maximum of the prefix. Whenever $a[i]$ is smaller than that maximum, set $r=i$. Scan right to left maintaining the minimum of the suffix; whenever $a[i]$ is larger, set $l=i$. If no violation occurs, return $[-1,-1]$. Every right-hand violation requires any repair interval to reach at least that far right, and every left-hand violation forces it at least that far left. Outside these endpoints the elements have no inversion with later or earlier elements respectively. Sorting the middle therefore fixes all remaining inversions. Two scans cost $O(n)$ time and $O(1)$ space. Use strict inequalities so equals are not violations.

Variation review

For reversal, compare with a sorted copy, reverse the interval between the first and last mismatches, and verify the entire result. If a successful reversal has matching positions at its outside edges, those edges can be trimmed in symmetric pairs; thus the mismatching interval is sufficient to test. $[1,3,2,4,3,5]$ can be repaired by sorting $[1,4]$, but reversing it gives $[1,3,4,2,3,5]$, which is still unsorted. Sorting destroys internal disorder; reversal only changes its direction.

Exit ticket and repair

Why might the first and last adjacent descents fail to identify the full interval? Explain why your final endpoints are necessary as well as sufficient.

If the explanation is still unclear: Trace running maxima and minima on $[1,3,2,2,4]$, marking the evidence that forces each endpoint.

Log: main attempted __; independent solve __; assisted solve __; partial __; variation status __; insight assistance __; minutes to first useful claim __; total minutes __.